



SOFTWARE TESTING SUPPLIMENT & PROJ.1: JUNIT INSTRUCTOR (ECLIPSE)

2015/10/7

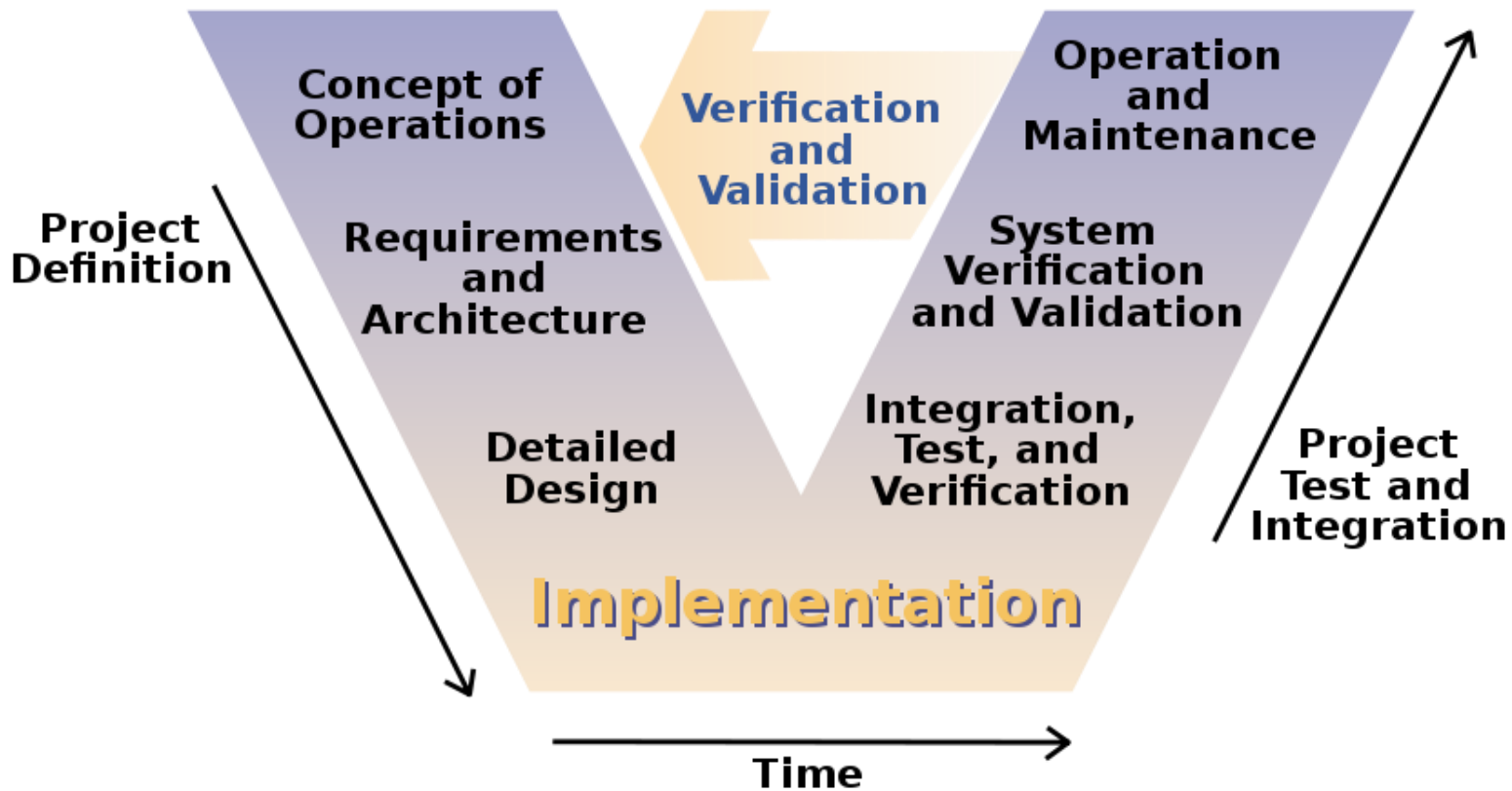
Speaker: TA: Hung Shiu-Jr.

Adviser: Farn Wang

OUTLINE

- Industrial software testing project (V model)
- IEEE 829 (software testing process standard)
- Industrial sample (Bluetooth communication)
- Project 1 sample

V MODEL



V MODEL

- Operation and Maintenance: decide the testing flow, includes languages, report forms...etc.
- System verification and validation: decide testing specifications.
- Integration testing and verifications: translate testing spec. to testing procedure.
- Implementation: design auto/non-auto test scripts. (For RD, also implement unit test.)

IEEE 829

- Test design spec. (based on design spec.)
- Test plan (collection of test items)
- Test cases (test procedures of test items)
- Tools and outputs (languages)
- Simulators (scripts or programs)
- Static and dynamic generators (random testing samples: positive, negative, stress...etc)
- Error and execution logs
- Reports and actions

PROJECT 1

- Eclipse (create a java project and class)

<https://www.youtube.com/watch?v=v2F49zLLj-8>

- Eclipse (create JUnit test cases)

<https://www.youtube.com/watch?v=I8XXfgF9GSc>

- Junit test methods
- Junit assert types
- Sample demonstration
- References

JUNIT TEST METHODS

- `@Test public void method()`
- `@Test(timeout = int)`
- `@Before public void method()`
- `@After public void method()`
- `@BeforeClass public static void method()`
- `@AfterClass public static void method()`
- `@Ignore`

JUNIT ASSERT TYPES

- `assertTrue(string message, boolean condition)`
- `assertFalse(string message, boolean condition)`
- `assertEquals(string message, expected, actual)`
- `assertEquals(string message, expected, actual, tolerance)`
- `assertSame(string message, expected, actual)`
- `assertNotSame(string message, expected, actual)`

SAMPLE DEMONSTRATION

- Test Requirements:

- Test a given prime number.
- Test a given non-prime number.
- Test a given negative integer.
- Test a given large prime number to check timeout tolerance.
- Test a given integer to count it's number of prime factors.
- Test a given negative integer.
- Test a given large number to check timeout tolerance.

- Test cases:

- positive test
- negative test
- stress test for negative input
- stress test
- positive test
- negative test
- stress test

REFERENCES

- [1] <http://www.vogella.com/tutorials/JUnit/article.html>
- [2] <http://junit.org/>
- [3] <http://eclipse.org/>